

Project Interim Report
On
File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26



Submitted By
ARYAN KUMAR
ERP Id-10289.

**Bachelor of Technology in Computer Science &
Engineering in association with GOOGLE**

Under the Guidance of
SHIVANSH MEHTA

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Index

Sr No.	Topic	Page No.
1	Abstract	1
2	Introduction	2
3	Scope	3
4	System Requirements	4
5	Methodology	5
6	Flow Chart	6
7	Implementation	7
8	Gantt Chart	8
9	Conclusion	9
10	Reference	10

Abstract :

The Library Management System is a software application developed using Object-Oriented Programming (OOP) principles in Java to efficiently manage library operations such as storing, searching, issuing, and returning books. The main objective of this project is to provide a simple and structured way to handle book records while demonstrating the practical implementation of core OOP concepts including classes, objects, encapsulation, constructors, and arrays of objects.

In this system, a "Book" class is designed to represent individual books with attributes such as title, author, and availability status. A "Library" class is implemented to manage a collection of books using an array structure. The system allows users to perform essential operations like searching for a book by its title, issuing a book if it is available, and returning a previously issued book. The availability status of each book is dynamically updated based on user actions, ensuring accurate tracking of resources.

The program emphasizes encapsulation by keeping data members private and providing controlled access through getter and setter methods. Method-based state changes are used to update book availability, ensuring that the logic remains organized and maintainable. Additionally, the use of arrays of objects helps in efficiently managing multiple book records within the system.

This project simulates real-world library operations and helps in understanding how object composition works, where the Library class contains multiple Book objects. It also improves problem-solving skills and code modularity by dividing the system into logical components.

Overall, the Library Management System serves as a foundational project for beginners to understand how software systems are designed using OOP concepts and how real-world problems can be solved through structured programming techniques.

Introduction :

In the modern digital era, managing information efficiently has become a fundamental requirement in every organization. Libraries, being important centers of knowledge and learning, require systematic methods to manage their resources effectively. Traditionally, library operations such as maintaining book records, tracking availability, issuing, and returning books were performed manually. This manual approach is time-consuming, error-prone, and difficult to manage as the number of books and users increases. To overcome these challenges, automated systems like the Library Management System are essential.

The Library Management System is designed using Object-Oriented Programming (OOP) principles to provide a structured and efficient way of managing library operations. This project focuses on implementing core OOP concepts such as classes, objects, encapsulation, and constructors to simulate real-world functionality. The system is built around two main components: the Book class and the Library class. The Book class represents individual book entities with attributes like title, author, and availability status, while the Library class manages a collection of books and handles operations performed on them.

The system allows users to search for books, issue available books, and return previously issued books. It ensures that the availability status of each book is updated dynamically, providing accurate and reliable information. By using arrays of objects, the system efficiently stores and manages multiple book records.

This project not only simplifies library management but also helps students understand how real-world problems can be solved using programming techniques. It enhances logical thinking, improves code organization, and demonstrates the practical application of OOP concepts in software development. Overall, the Library Management System serves as a foundational step toward building more advanced and scalable applications.

The main objectives of this project are:

To develop a system that can manage book records efficiently

To track the availability and issuing status of books

To reduce manual work and errors in library management

To implement and demonstrate Object-Oriented Programming concepts

To provide a simple and interactive user interface

Overall, this project provides a practical understanding of how programming concepts can be applied to solve real-world problems in an efficient manner.

Scope:

The scope of the Library Management System project is to design and implement a basic software solution that simplifies and automates core library operations using Object-Oriented Programming principles. This system focuses on managing a collection of books and performing essential functions such as searching, issuing, and returning books in an efficient and organized manner.

The project is limited to a small-scale implementation suitable for academic purposes, where a fixed number of books are stored using an array of objects. It allows users to view available books, search for specific titles, issue books if they are available, and return books after use. The system dynamically updates the availability status of each book, ensuring accurate tracking of library resources.

This project primarily demonstrates the practical application of OOP concepts such as encapsulation, class design, constructors, and method-based state changes. It also highlights how real-world problems like resource management can be solved using structured programming techniques. The scope includes building a console-based application without the use of advanced technologies such as databases or graphical user interfaces.

Although the current system is simple, it provides a strong foundation for future enhancements. The project can be extended to include features such as user authentication, fine calculation, book reservation, and integration with databases for large-scale storage. It can also be upgraded to a graphical user interface or web-based application for better usability.

Overall, the scope of this project is to create a functional and educational model of a library system that helps in understanding software development concepts while solving a real-world problem in a simplified manner.

System Requirements:

Hardware: Computer with 4GB RAM

Software: Java JDK, IDE (VS Code/Eclipse/Intelligent)

Minimum System Requirements:

Operating System: Windows 7 or later / Linux / macOS

Processor: Intel Core i3 or equivalent

RAM: Minimum 2 GB (4 GB recommended)

Storage: At least 500 MB free space

Software: Java JDK (version 8 or above)

IDE: Any basic editor like VS Code, Eclipse, or Notepad++

Methodology:

The methodology of the Library Management System project follows a structured approach to design, develop, and test the application using Object-Oriented Programming principles. The development process begins with understanding the problem requirements, which include managing book records, tracking availability, and handling issue and return operations efficiently.

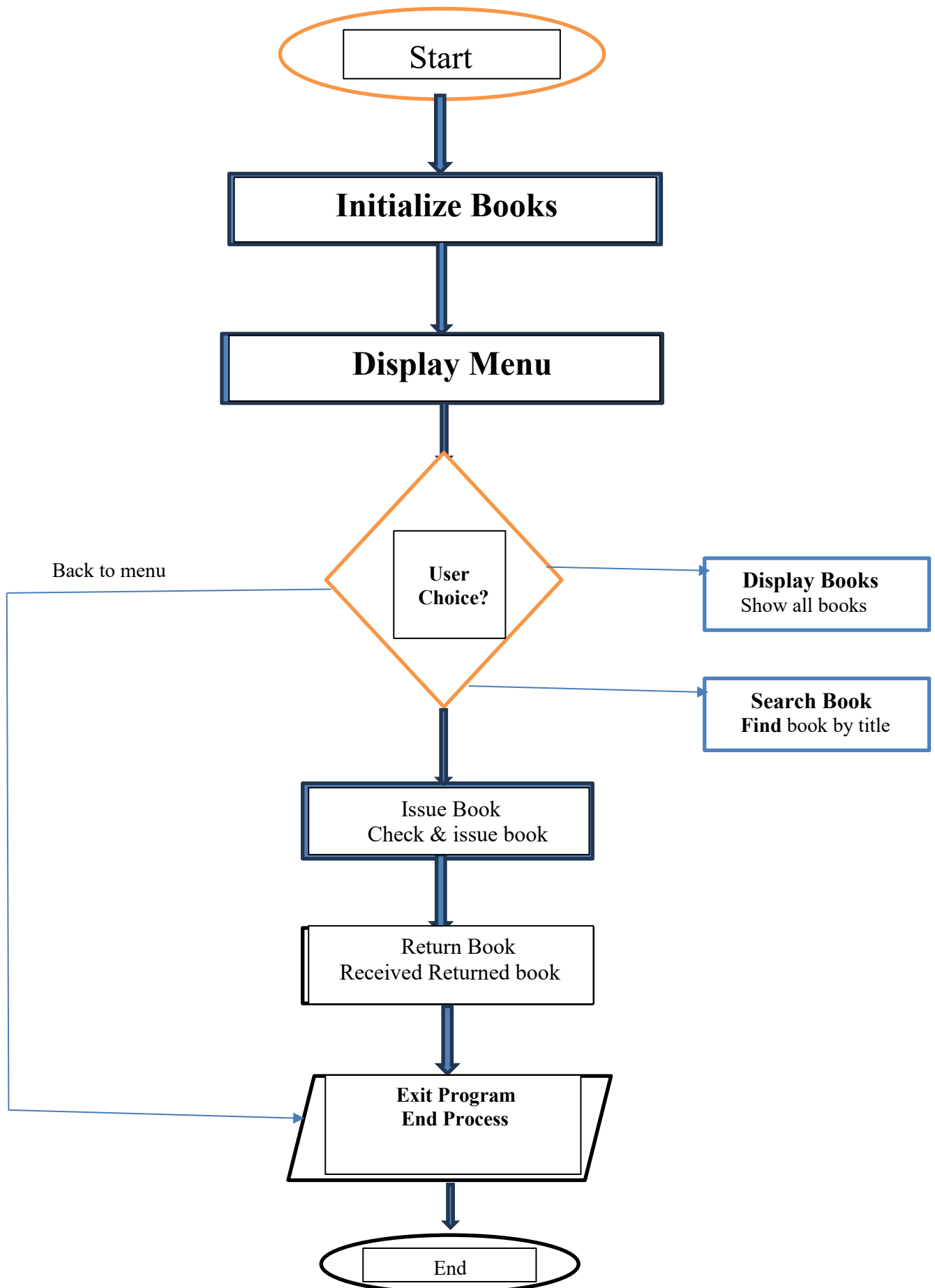
The first step involves designing the system structure by identifying the main classes required. Two primary classes are created: the Book class and the Library class. The Book class is designed to represent individual book entities with attributes such as title, author, and availability status. Encapsulation is applied by declaring these attributes as private and providing public getter and setter methods to access and modify them.

The second step focuses on implementing the Library class, which manages a collection of books using an array of objects. The array is initialized with sample book data to simulate a real library environment. This class contains methods to perform key operations such as displaying all books, searching for a book by title, issuing a book, and returning a book.

The third step includes implementing the logic for each operation. The search functionality scans the array to find a matching book title. The issueBook method checks availability before updating the status, while the returnBook method restores the availability of issued books. These operations ensure proper state management of each book.

Finally, the system is tested through the main method by creating a Library object and invoking different methods to verify functionality. This step ensures that all features work correctly and handle different scenarios effectively. Overall, the methodology emphasizes modular design, code reusability, and clear logical flow.

FlowChart:



Implementation:

Implementation

The project mainly uses two classes: Book and Library.

The Book class represents a single book in the library. It stores basic details like the title, author, and whether the book is available or not. The availability is managed using a boolean value. When a book is issued, its availability becomes false, and when returned, it becomes true again. This class also provides methods to access and update these values, ensuring proper data control.

The Library class manages a collection of books using an array of Book objects. It acts as the main controller of the system. This class contains methods to perform different operations such as:

- Displaying all books with their status
- Searching for a book by title
- Issuing a book only if it is available
- Returning a book and updating its status

The logic works by looping through the array to find the required book and then performing the appropriate action.

The main class is used to test the system. It creates a Library object and calls different methods to simulate real library operations like searching, issuing, and returning books.

Available Books:

Java Basics - Available

Data Structures - Available

Search: Java Basics

Book Found

Issuing Book: Java Basics

Book issued successfully

Issuing Book: Java Basics

Book is already issued

Returning Book: Java Basics

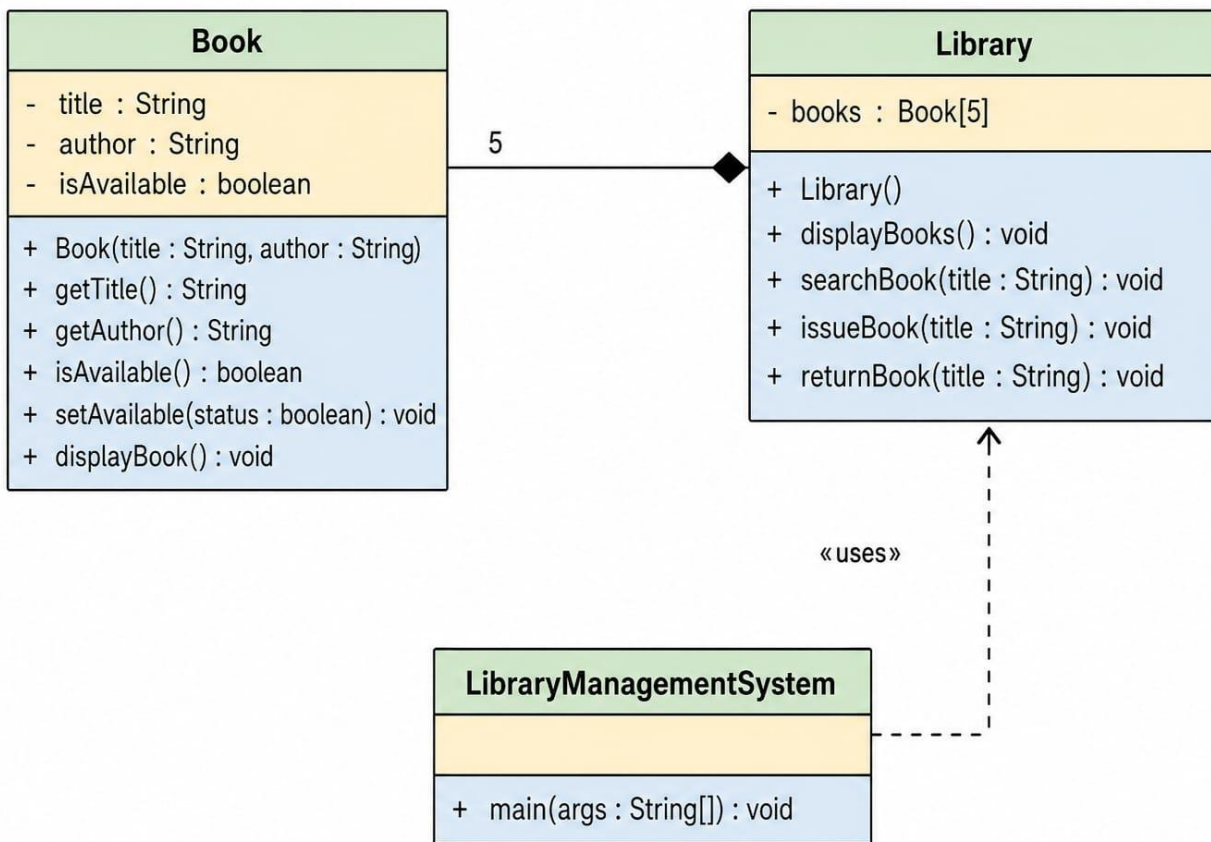
Book returned successfully

This project can be improved in several ways to make it more advanced and practical:

- Add a user login system for students and admin
- Use a database (MySQL) instead of arrays for large data storage
- Develop a GUI (Graphical User Interface) using Java Swing or JavaFX
- Add fine calculation for late returns
- Implement book reservation system
- Allow adding and deleting books dynamically
- Convert it into a web-based application

These improvements will make the system more realistic and suitable for real-world use.

Class Diagram:



A class diagram is a visual representation of classes in an object-oriented system. It shows attributes, methods, and relationships like association, inheritance, and dependency. It helps developers understand system structure, design efficiently, and communicate ideas clearly during software development and maintenance processes. It also supports planning, debugging, and documentation activities.

GANTT CHART:

A Gantt chart is a visual tool used to plan and manage the different stages of a project. It helps in organizing tasks, assigning timelines, and tracking progress effectively.

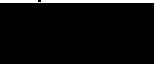
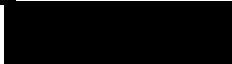
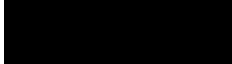
1. Planning & Structure:

The project is divided into smaller tasks such as requirement analysis, designing classes, coding, testing, and documentation. Each task is arranged in a proper sequence so that the development process becomes smooth and systematic. Even though the project is small, planning ensures that every step is completed correctly.

2. Time Management

The Gantt chart clearly shows when each task starts and when it should be completed. This helps in managing time efficiently and avoiding delays. It also reduces last-minute work pressure by ensuring that tasks are completed on time.

3. Sample Gantt Chart (Timeline)

Task	Duration	Week 1	Week 2	Week 3	Week 4
Requirement Analysis	2 days				
Class Design (Book,Library)	3 days				
Coding	5 days				
Testing	3 days				
Documentation	3 days				

The shaded blocks represent the time period during which each task is performed.

Overall, the Gantt chart helps in better planning, time management, and successful completion of the project.

Code ScreenShot

```
// Book Class
class Book { 18 usages
    private String title; 3 usages
    private String author; 3 usages
    private boolean isAvailable; 4 usages

    // Constructor
    @Contract(pure = true)
    public Book(String title, String author) { 5 usages
        this.title = title;
        this.author = author;
        this.isAvailable = true; // default available
    }

    // Getter methods
    public String getTitle() { 3 usages
        return title;
    }

    public String getAuthor() { no usages
        return author;
    }

    public boolean isAvailable() { 2 usages
        return isAvailable;
    }

    // Setter method for availability
    public void setAvailable(boolean status) { 2 usages
        this.isAvailable = status;
    }
}
```

```
// Display book details
public void displayBook() { 2 usages
    System.out.println(title + " by " + author + " - " +
        (isAvailable ? "Available" : "Issued"));
}

// Library Class
class Library { 2 usages
    private Book[] books; 10 usages

    // Constructor
    public Library() { 1 usage
        books = new Book[5];

        // Initialize sample books
        books[0] = new Book(title: "Java Basics", author: "James Gosling");
        books[1] = new Book(title: "Data Structures", author: "Mark Allen Weiss");
        books[2] = new Book(title: "Operating Systems", author: "Galvin");
        books[3] = new Book(title: "Computer Networks", author: "Tanenbaum");
        books[4] = new Book(title: "DBMS", author: "Korth");
    }

    // Display all books
    public void displayBooks() { 2 usages
        System.out.println("\nAvailable Books:");
        for (Book book : books) {
            book.displayBook();
        }
    }
}
```

```

public void searchBook(String title) { 1 usage
    boolean found = false;

    for (Book book : books) {
        if (book.getTitle().equalsIgnoreCase(title)) {
            System.out.println("\nBook Found:");
            book.displayBook();
            found = true;
            break;
        }
    }

    if (!found) {
        System.out.println("\nBook not found.");
    }
}

// Issue Book
public void issueBook(String title) { 2 usages
    for (Book book : books) {
        if (book.getTitle().equalsIgnoreCase(title)) {

            if (book.isAvailable()) {
                book.setAvailable(false);
                System.out.println("\nBook issued successfully.");
            } else {
                System.out.println("\nBook is already issued.");
            }
            return;
        }
    }
}

```

```

        System.out.println("\nBook not found.");
    }

    // Return Book
    public void returnBook(String title) { 1 usage
        for (Book book : books) {
            if (book.getTitle().equalsIgnoreCase(title)) {

                if (!book.isAvailable()) {
                    book.setAvailable(true);
                    System.out.println("\nBook returned successfully.");
                } else {
                    System.out.println("\nBook was not issued.");
                }
                return;
            }
        }
        System.out.println("\nBook not found.");
    }
}

// Main Class
public class LibraryManagementSystem {
    public static void main(String[] args) {

        Library library = new Library();

        // Display books
        library.displayBooks();
    }
}

```

```
// Main Class
public class LibraryManagementSystem {
    public static void main(String[] args) {

        Library library = new Library();

        // Display books
        library.displayBooks();

        // Search book
        library.searchBook( title: "Java Basics");

        // Issue book
        System.out.println("\nIssuing Book: Java Basics");
        library.issueBook( title: "Java Basics");

        // Try issuing again
        System.out.println("\nIssuing Book: Java Basics");
        library.issueBook( title: "Java Basics");

        // Return book
        System.out.println("\nReturning Book: Java Basics");
        library.returnBook( title: "Java Basics");

        // Final list
        library.displayBooks();
    }
}
```

Conclusion:

The Library Management System project successfully demonstrates the practical application of Object-Oriented Programming (OOP) concepts using Java. The system provides a simple yet effective solution for managing library operations such as storing book details, searching for books, issuing available books, and returning issued books. Through this project, important programming concepts like classes, objects, constructors, encapsulation, and arrays of objects have been clearly implemented and understood.

The use of a Book class to represent individual books and a Library class to manage a collection of books highlights the concept of object composition. The system ensures proper state management by dynamically updating the availability status of each book during issue and return operations. This reflects how real-world systems maintain and update data efficiently.

Additionally, the project emphasizes clean code structure, modular design, and logical flow, which are essential for building maintainable software applications. Even though the system is console-based and limited in scale, it provides a strong foundation for understanding how software solutions can be developed for real-life problems.

Overall, this project not only strengthens programming skills but also enhances problem-solving abilities. It serves as a stepping stone for developing more advanced applications by incorporating features such as graphical interfaces, database integration, and user authentication. The successful implementation of this system proves that OOP concepts can be effectively used to design organized, scalable, and efficient software solutions.

REFERENCE:

Books

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.

Websites

[2] Oracle, "Java Documentation," Oracle. [Online]. Available: <https://docs.oracle.com>

Conference Paper

[3] S. Kumar, "AI in Education," in Proc. IEEE Conf. on AI, 2021, pp. 120-125.

Journal Article

[4] A. Sharma and R. Gupta, "Machine Learning Techniques," International Journal of AI, vol. 5, no. 2, pp. 45-50, 2020.